AAA: Authentication, Authorization, and Accounting

IE3122: Network Security | Week 10

Three separate questions, three separate answers

Today's Agenda

- What AAA actually is, and why it's three separate functions, not one
- Authentication: the factors that prove who you are
- Authorization: privilege levels, command authorization, and the four access control models, DAC, MAC, RBAC, and RuBAC
- Accounting: what gets tracked, and why it matters even when access was properly granted

Learning Outcomes for Today

LO3

- Analyse security requirements and recommend appropriate technologies.

LO4

- Evaluate different security technologies, compare them, and identify appropriate use cases.

- *By the end of this session you should be able to explain what each of the three A's actually controls, and recommend the right access control model for a given scenario.*

Foundations

Three genuinely separate questions

What Is AAA?

- Authentication answers who are you, verifying that an entity genuinely is who it claims to be.
- Authorization answers what are you allowed to do, determining which resources and actions an authenticated entity can actually access.
- Accounting answers what did you actually do, recording activity for auditing, billing, or investigation after the fact.
- These are three distinct security functions working together, not three names for the same check.

Why Three Separate Functions, Not One

- You can be authenticated without being authorized for everything. A verified employee has proven who they are, but that alone doesn't mean they can reach every system in the building.
- Authorization is meaningless without authentication first. You cannot decide what someone is allowed to do until you know who they actually are.
- Accounting happens regardless of the authorization outcome. Both successful actions and denied attempts get recorded, since a pattern of failed access attempts is itself valuable security information.
- Merging these into one blunt pass/fail check loses exactly this granularity, and is why real systems keep them as three deliberately separate stages.

AAA as a Modular Framework

- Recall Week 9: IPsec was built as a modular framework, a menu of interchangeable choices for each of several functions, not one fixed protocol.
- AAA is built the same way. The authentication method (password, certificate, biometric, multi-factor), the authorization model (covered in detail shortly), and the accounting mechanism (local logging or centralised) are all independently selectable.
- A real AAA deployment is assembled by picking one option from each of these three rows, exactly like building an IPsec configuration from its own framework table.

Authentication

Proving who you actually are

Something You Know

- Mechanism: the user provides a secret only they should know, a password, PIN, or answer to a security question, which the system checks against a stored reference value.
- Examples: passwords, PINs, passphrases, and security questions such as a first pet's name.
- Use cases: the default, baseline method for almost every system, website logins, device unlock codes, and initial account setup.
- Strengths: cheap and simple to implement, needs no special hardware, universally understood, and can be changed instantly if compromised.
- Weaknesses: vulnerable to phishing, guessing, shoulder-surfing, and keylogging. Users routinely choose weak or reused passwords across many sites, undermining the whole model even when the mechanism itself is sound.

Something You Have

- Mechanism: the user proves possession of a physical or digital item, and the system verifies a code that item generates or receives.
- Examples: hardware security keys, smart cards, RSA-style tokens, SMS one-time codes, and authenticator apps generating time-based codes.
- Use cases: banking cards paired with a PIN, corporate VPN access via a token, two-factor authentication on sensitive accounts, and badge-based building access.
- Strengths: cannot be guessed or phished the way a password can, since an attacker needs to physically obtain or clone the item itself.
- Weaknesses: the item can be lost or stolen, and SMS-based codes specifically can be intercepted through SIM-swapping attacks. Replacing a lost token has real cost and logistics.

Something You Are: Physical Biometrics

- Mechanism: measures a fixed, largely unchanging anatomical characteristic of the body, captured by a scanner or sensor and compared against a stored enrolment template.
- Examples: fingerprint ridge patterns, facial geometry, iris and retina patterns, hand geometry, and DNA matching.
- Use cases: smartphone unlock, border control and passport e-gates, high-security facility access, and forensic identification.
- Strengths: extremely high uniqueness between individuals, and stable over long periods of time, a person's fingerprint or iris pattern doesn't meaningfully change year to year.
- Weaknesses: needs specialised hardware, can be disrupted by physical conditions such as a cut affecting a fingerprint or cataracts affecting an iris scan, and if the trait itself is ever compromised, it can never be reissued, unlike a password or even a physical token.

Something You Are: Behavioural Biometrics

- Mechanism: measures patterns in how a person performs an action over time, rather than a fixed physical trait, compared against a behavioural baseline built through observation.
- Examples: keystroke dynamics (typing rhythm and pressure), gait analysis, signature dynamics (the speed and rhythm of signing, not just the resulting image), and speech cadence.
- Use cases: continuous authentication running in the background during a session, fraud detection during a banking transaction, and ongoing workplace session verification.
- Strengths: can authenticate continuously throughout a session rather than only once at login, making it far harder for an attacker who got past the initial login to stay undetected, and it's genuinely difficult to consciously imitate someone else's natural patterns.
- Weaknesses: less mature and standardised than physical biometrics, with higher false-positive and false-negative rates, since normal behaviour naturally varies day to day. Behavioural traits are also less stable over time than physical ones, changing with stress, fatigue, injury, or a different device.

Multi-Factor Authentication

- Mechanism: combines two or more factors from genuinely different categories above, not two instances of the same category. Two different passwords is still only one factor type, repeated.
- Examples: a password combined with an authenticator app code; a fingerprint combined with a PIN; a smart card combined with biometric verification for high-security facilities.
- Use cases: online banking, corporate remote access and VPN (recall Week 9), cloud service logins, and any system protecting genuinely sensitive data or high-value transactions.
- Strengths: dramatically reduces the risk that one compromised factor leads to full account takeover, and is increasingly treated as a baseline expectation rather than an optional extra.
- Weaknesses: adds friction to login that can hurt adoption if handled poorly, and isn't perfectly immune either. SIM-swapping can still defeat SMS-based MFA specifically, and prompt bombing can trick a user into approving a push notification they never initiated.

Authentication Methods, Compared

Method	Example	Key Strength	Key Weakness
Something You Know	Password, PIN	Cheap, simple, no hardware needed	Phishable, guessable, often reused
Something You Have	Hardware token, authenticator app	Cannot be guessed or phished remotely	Can be lost, stolen, or SIM-swapped
Are, Physical	Fingerprint, facial recognition	High uniqueness, stable over time	Permanent if compromised; needs hardware
Are, Behavioural	Keystroke dynamics, gait	Enables continuous authentication	Less stable; higher false-positive rate
Multi-Factor	Password plus authenticator app	Defeats a single compromised factor	Adds friction; vulnerable to prompt bombing or SIM-swapping

Authorization

Deciding what an authenticated identity can actually do

Privilege Levels and Command Authorization

- Cisco IOS uses a numeric privilege scale from 0 to 15. Level 0 has almost no access, level 1 is basic user EXEC mode, and level 15 is full privileged, administrative access.
- Command authorization goes further than simply logged in or not. It controls exactly which specific commands an authenticated user can run once they're in, for example allowing a junior technician to run show commands while denying configure terminal entirely.
- This is the practical, device-level version of authorization's core question: once someone's identity is confirmed, exactly how much power do they actually have?

Access Control Models: Overview

- Beyond privilege levels on one device, four general models exist for deciding who actually controls access decisions across a system, and how.
- DAC: the resource's own owner decides. MAC: a central authority decides, using fixed classification labels. RBAC: access is tied to a job role. RuBAC: access is decided by conditional rules evaluated at the moment of the request.
- These models are genuinely independent of which protocol carries the decision. RADIUS and TACACS+, covered in Deck B, are how a decision gets communicated. These four models are what actually gets decided in the first place.

DAC, Discretionary Access Control

- The owner of a resource decides who else can access it, and can grant or revoke that access at their own discretion, at any time.
- Example: standard Windows and Linux file permissions. If you create a file, you personally decide who else can read or write it.
- Advantage: highly flexible, since decisions are made by whoever actually knows the resource best.
- Disadvantage: control is scattered across every individual owner, making a single, consistent, organisation-wide policy difficult to enforce or audit.

MAC, Mandatory Access Control

- A central authority sets fixed classification labels, such as Top Secret, Secret, Confidential, and Unclassified, and access is strictly determined by comparing a user's clearance to a resource's label.
- Individual users, even a file's own creator, cannot override or change these labels themselves.
- Example: military and government classified systems, and SELinux's enforcement model.
- Advantage: highly consistent, since no individual can accidentally or deliberately weaken the policy. Disadvantage: inflexible, with real administrative overhead to manage classifications correctly.

RBAC, Role-Based Access Control

- Access is tied to a user's job role within the organisation, not to them individually and not to any resource owner's personal discretion.
- Example: an HR Manager role automatically gets access to personnel files; a Sales Representative role gets access to the CRM but not to HR systems at all.
- Advantage: manages well at scale. When someone changes jobs, you change their role assignment once, rather than touching every individual permission they hold.
- This is the dominant model in most enterprise networks, and the one that most directly maps onto AAA's authorization function in a typical business deployment.

RuBAC, Rule-Based Access Control

- Access is decided by evaluating if-then conditional rules at the moment of the request, rather than a fixed, pre-assigned label or role.
- Example conditions: time of day, only allowing access during business hours; source location or IP address; device type; or combinations of these evaluated together.
- Advantage: genuinely context-aware, able to adapt access dynamically as circumstances change.
- Recall Week 7: an ACL is itself a rule-based access control mechanism, evaluated top to bottom, condition by condition. RuBAC is that same logic applied to authorization generally, not just packet filtering.

The Four Access Control Models, Compared

Model	Who Decides	Flexibility	Typical Use Case
DAC	The resource's own owner	High, but inconsistent across owners	General-purpose file systems (Windows, Linux)
MAC	A central authority, via fixed labels	Low, but highly consistent	Military and government classified systems
RBAC	The user's organisational role	Moderate, scales well	Enterprise networks, most business systems
RuBAC	Conditional rules evaluated at request time	High, genuinely context-aware	ACLs, firewalls, time or location-restricted access

Accounting

What actually happened, and proving it afterward

Session Accounting

- Mechanism: records the start and stop time of an authenticated session, when a user logged in, when they logged out or timed out, and by calculation, the total duration.
- Examples: a log entry showing a specific user logged into a router at a given time and the session ended some minutes later, exactly what Cisco's aaa accounting exec records.
- Use cases: a basic audit trail of who accessed a system and for how long, capacity planning around concurrent sessions at peak times, and spotting unusual login timing, such as an admin account logging in at 3am when it normally only appears during business hours.
- Strengths: simple and low overhead to record, just two timestamps per session, giving a genuinely useful audit trail with minimal system impact.
- Weaknesses: tells you who was logged in and for how long, but nothing about what they actually did during that session. A clean login and logout record says nothing about whether the session was used for legitimate work or something malicious.

Command Accounting

- Mechanism: records every individual command an authenticated user actually executes during their session, building a detailed, command-by-command audit trail.
- Examples: TACACS+'s aaa accounting commands logging each CLI command a network administrator runs, for instance entering configuration mode, selecting an interface, then shutting it down, each as its own timestamped entry.
- Use cases: detailed forensic investigation after an incident, reconstructing exactly what an administrator did; compliance requirements demanding proof of precisely what changed and by whom; and simply deterring insider misuse, since users know every command is logged.
- Strengths: this is the real audit trail. It directly enables non-repudiation, since you can prove exactly which action a specific identity performed, not just that they were logged in at some point.
- Weaknesses: meaningfully higher overhead than session accounting, since every command must be captured and stored, not just two timestamps. This granularity is one of TACACS+'s defining advantages over RADIUS, recall Deck B, and it generates real volumes of log data that need proper storage and review to actually be useful.

Resource Accounting

- Mechanism: records the actual resources consumed during a session, data volume transferred, bandwidth used, or connection time.
- Examples: an ISP's accounting records historically tracking how many megabytes a customer used in a billing period; a corporate VPN gateway logging how much bandwidth each remote user consumed.
- Use cases: historically the basis for usage-based ISP billing; today used more for capacity planning and usage monitoring, and for spotting anomalies, since a sudden, unusually large data transfer from one account can be a genuine signal of data exfiltration.
- Strengths: gives visibility into the scale of activity, not just its existence, useful for both billing and security monitoring.
- Weaknesses: doesn't say what was actually done, only how much. A large transfer could be a legitimate backup job or genuine data theft, and resource accounting alone can't tell the two apart without session or command context alongside it.

Accounting Types, Compared

Type	What It Records	Example	Best For
Session	Login and logout time, duration	User logged in 09:14, out 09:47	Basic audit trail, capacity planning
Command	Every command executed	User ran configure terminal at 09:15	Forensic investigation, non-repudiation
Resource	Data volume, bandwidth, connection time	ISP billing records, VPN bandwidth logs	Billing, usage monitoring, anomaly detection

Accounting and Non-Repudiation

- Recall Week 2: non-repudiation means a party cannot later deny having performed an action.
- Accounting is what actually makes non-repudiation possible in practice. Without a recorded log tying a specific action to a specific authenticated identity, there is no way to prove afterward who did what.
- This is why accounting matters even for successful, fully authorized actions, not just failed or denied ones. A properly authorized action can still need to be proven later, in an audit or an investigation.

Summary

- Authentication, Authorization, and Accounting are three separate, sequential functions. You cannot authorize before authenticating, and accounting happens regardless of the outcome.
- AAA is a modular framework, echoing IPsec's design from Week 9: an authentication method, an authorization model, and an accounting mechanism, each independently chosen.
- DAC, MAC, RBAC, and RuBAC all answer the same question, who actually controls an access decision, in four genuinely different ways, with different trade-offs between flexibility and consistency.
- Accounting isn't just a record of failures. It's what turns an authorized action into something that can still be proven after the fact.

Coming Up

- Deck B, next: how AAA is actually deployed, local versus centralised, device administration versus network access, RADIUS versus TACACS+ in full detail, and Cisco configuration.
- Tutorial worksheet: study-notes-style questions covering today's three functions and the four access control models. Use it to prep for the mid-semester test.

References

- Santos & Stuppi, CCNA Security 210-260 Official Cert Guide, 1st Ed., Cisco Press.
- Kaufman, Perlman, Speciner & Perlner, Network Security: Private Communication in a Public World, 3rd Ed., Prentice Hall.
- Cisco Networking Academy, CCNA Security curriculum (AAA chapters).